论文-18 · 压缩的形状：叙事与归档

何淇 · 与 DeepSeek 协作

数字生命系列 · 实验篇 8（内部编号第 18 篇）

日期：2026-10-24

摘要

大模型的上下文压缩是一种"叙事摘要"：按时间顺序把发生的对话写成一段文字。人脑的压缩不是这样：它把事件归到类别里、把类别归到更高的抽象层里、再保留事物之间的关系——压缩的结果是一张"分类 + 关系"的图，而不是一段故事。

本文做一个对照实验：同样约 300 字符的记忆容量，三种存法——叙事式（旧优先）、叙事式（新优先）、结构化归档（分类层次 + 属性表 + 关系边），然后问三类问题：属性（这实体什么状态）、分类（属于哪一类）、关系（两者有关系吗）。

结果：结构化归档在全部三类题上都是满分；两种叙事式各丢一半——旧优先的保住属性和分类（各 0.83），关系只拿到 0.33；新优先的保住关系（1.00），属性掉到 0.25、分类 0.50。也就是说，叙事式压缩无论怎么权衡，都只能覆盖一部分；结构化压缩用更少字符（243 对 298）覆盖了全部。

解读：叙事式压缩是线性的，容量有限时必然丢一段；结构化压缩把"分类、属性、关系"分开存，信息密度高得多。这对应大模型 checkpoint 压缩的已知短板：过程记得住，层级与关系最容易丢。全部是合成任务，不涉及真实模型。

一、问题

压缩的意思是用更少的空间保留更多的信息。但"保留什么"取决于压缩的形状。

大模型的上下文压缩是叙事式的：把对话按时间顺序写成摘要。人脑的压缩是归档式的：事件被切开、归类，类别被放到抽象层里，关系被保留下来。

这两种形状有什么可测的差别？如果容量一样，哪一类信息会丢？

二、设定

生成一份"项目记录"：4 个类别（工具、人员、实验、结论），每类 5 个实体，共 20 个；每个实体有一个状态属性（活跃、冻结、待定）；另有 12 条关系（"使用"或"导致"）。

三种记忆，容量都约 300 字符：

- A1 叙事式·旧优先：按时间顺序写句子，写满为止——保住早期事实，后面的关系写不进去；

- A2 叙事式·新优先：先写关系、再写近期实体，写满为止——保住关系，早期实体写不进去；

- B 结构化归档：分类层次（哪类有哪几个）、属性表（每个实体的状态，压缩成单字）、关系边（谁对谁做什么）。

然后问三类题，各 12 道：属性题（这实体当前什么状态）、分类题（属于哪一类）、关系题（这两个实体之间有关系吗）。答案只能从各自的记忆里检索。

三、结果

总体正确率：A1 是 0.67，A2 是 0.58，B 是 1.00。

分题型看：

- 属性题：A1 0.83，A2 0.25，B 1.00；

- 分类题：A1 0.83，A2 0.50，B 1.00；

- 关系题：A1 0.33，A2 1.00，B 1.00。

字符数：A1 用了 298，A2 用了 296，B 只用了 243。

四、解读

第一，叙事式压缩是线性的。记忆只能从头写到尾，容量有限就必然丢一段：要么丢关系，要么丢早期事实。A1 与 A2 就是这两种取舍，各有一个短板。

第二，结构化压缩的密度高得多。分类、属性、关系是三种不同结构，分开存就不互相挤占；同样容量下它覆盖了全部信息，还用得更少。

第三，这对应大模型 checkpoint 压缩的短板。现在的压缩只做叙事摘要，所以"发生了什么"记得住，而"属于哪一层、和什么有关系"最容易丢掉。人脑恰好相反：它先记住结构和关系，具体细节反而会忘。

五、对记忆设计的含义

如果这个对照成立，那么压缩的产物不应该只是一段摘要，而应该是两样东西：

- 一段叙事摘要：保留时间线与因果（"发生了什么"）；

- 一份结构化归档：实体表、分类层次、关系边（"属于哪里、和谁有关"）。

两者配合检索：问过程用摘要，问层级与关系用归档。这个设计可以直接测：把同一段长对话按两种方式压缩，比较三类问题的召回率——本文的实验是这条路的玩具版。

六、局限

第一，任务全是合成的：实体、关系、题目都是程序生成的。

第二，结构化归档的"抽取成本"没有计入：实验里结构是直接给出的，真实系统要先从对话里抽实体与关系，这一步会出错、也有成本。

第三，真实的摘要模型可能比 A1/A2 都强（它可以选择重要的写、不写流水账），所以数字不能直接搬到真实系统；但"叙事 vs 结构化"的密度差不会因此消失。

七、与既有工作的关系

把记忆分成"语义记忆与情景记忆"是认知科学的老区分；大模型一侧，记忆机制、知识图谱增强、检索增强、长上下文记忆库都已有大量工作。

本文的不同只在于一个公平对照：同样字符容量下，把叙事式与结构化归档放在同一张卷子上，按题型分解看谁丢什么。结论不新，但它把"为什么 checkpoint 压缩会丢关系"变成了一件可以演示的事。

八、可复现

脚本：910_memory_compression.py（主对照）、911_memory_compression_v2.py（加"新优先"对照）。依赖 Python 3 标准库，无第三方依赖。

附：标注

- **[事实]** 三种记忆的召回数字：A1 0.67、A2 0.58、B 1.00（分题型见表）；B 用 243 字符、A 用约 296~298。

- **[解释]** "叙事式是线性的、结构化密度更高"——对数字的解读，不是定理。

- **[推测]** 把压缩产物设计成"摘要 + 归档"能提升层级与关系类召回——未在真实系统验证。

- **[不声称]** 不声称提出了新的记忆架构；索引、图谱、检索都是已有方向。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。